Medal Prediction in Olympic Athletes Using Ensemble Learning and Class Imbalance Strategies

Muhammad Syukri bin Abdul Rahman
Faculty of Engineering Environment and Computing
Coventry University
Coventry, England
abdulra22@uni.coventry.ac.uk

***Abstract*—This paper predicts Olympic medal outcomes using a 120-year athlete dataset. Logistic Regression, Random Forest, and XGBoost were tuned under two imbalance strategies: SMOTE and random undersampling. A stacking ensemble combined the three base learners. XGBoost with undersampling delivered the top results (F1 = 0.306, ROC AUC = 0.701). SHAP analysis identified Age and Event_encoded as the most influential features, followed by Height and Sex_encoded.**

Keywords—Olympic analytics, class imbalance, SMOTE, undersampling, XGBoost, ensemble learning, SHAP

I. INTRODUCTION

Predicting Olympic medalists with machine learning helps federations identify promising athletes early and allocate resources more effectively in elite sports. As historical sports data have grown, machine-learning methods have expanded into areas such as athlete-performance forecasting and injury-risk analysis [1], [2]. Yet medal prediction remains difficult because athletic performance is complex and medal outcomes are highly imbalanced.

Earlier studies often relied on single classifiers or simple regression techniques and did not fully address severe class imbalance [3]. Many also failed to exploit complete historical records. More recent work has begun to incorporate socio-economic and contextual factors, especially during unusual periods like the COVID-19 pandemic [4].

This study addresses these limitations by comparing three algorithms: Logistic Regression (LR), Random Forest (RF), and XGBoost (XGB). Demographic and event-related features were used by the models drawn from the full Olympic record from 1896 to 2016. To mitigate imbalance, the training data are resampled with both the Synthetic Minority Oversampling Technique (SMOTE) and random undersampling. A stacking ensemble then integrates the strengths of the tuned models.

All models were tested on a separate hold-out set containing every athlete from the 2016 Games. Evaluation metrics included accuracy, precision, recall, F1 score, and receiver-operating-characteristic area under the curve (ROC AUC). Feature importance was examined with SHapley Additive exPlanations (SHAP). The findings highlight the value of undersampled ensemble models and provide a solid basis for further research on Olympic medal prediction.

II. DATASET AND PROBLEM FORMULATION

The open Kaggle collection was used “120 Years of Olympic History: Athletes and Results” [5], which records every Summer Games from 1896 to 2016. The raw file lists 271 116 athlete-event entries, each with demographic details, event information, and medal status. Rows missing age, height, weight, sex, or sport were removed, leaving 206 165 complete cases for analysis.

Medal outcomes are converted to a binary label called MedalBinary, where 1 marks an athlete who won any medal and 0 marks one who did not. This choice keeps the problem focused on whether an athlete reached the podium and sidesteps the extra complexity of differentiating gold, silver, and bronze. After cleaning, only 14.6 percent of records belong to medalists, making the data strongly imbalanced.

To test the models realistically and avoid leakage, the study applied a temporal split. All athletes from the 2016 Games form the hold-out test set, while the training set contains entries from 1896 through 2012. Unique athlete identifiers make sure no individual appears in both sets. This approach follows best practice for time-dependent prediction tasks [6].

Fig. 1 shows the large gap between medalists and non-medalists, underlining the need for techniques that handle class imbalance before model training.

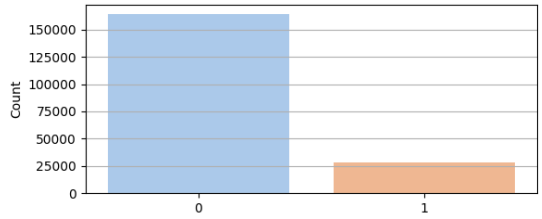


Fig. 1. Class distribution before resampling.

III. METHODS

A. Preprocessing and Feature Engineering

After removing rows with missing demographic fields, 206,165 complete cases remained. Event and Sex were label-encoded to produce Event_encoded and Sex_encoded. BMI was computed as weight divided by height squared to represent body composition more effectively than raw height or weight alone. The final feature set consisted of Age, Height, Weight, BMI, Event_encoded, and Sex_encoded, as summarised in Table I.

TABLE I. SELECTED FEATURES AND THEIR RATIONALE

Feature	Description	Rationale
Age	Athlete's age (years)	Correlates with physical maturity
Height	Height (cm)	Relevant for height-dependent events
Weight	Weight (kg)	Indicator of strength or power
BMI	Weight / Height²	Normalised body composition
Event_encoded	Encoded event type	Differentiates event context
Sex_encoded	Encoded gender	Captures gender performance effects

Although many event names imply gender (e.g., "Men's 100 metres"), others are mixed or ambiguous (e.g., "Mixed Doubles"), and label encoding removes these distinctions. To consistently capture gender-based performance differences, Sex was retained as a separate feature.

A chronological split separated the data: entries from 1896 to 2012 were used for training, and athletes from 2016 were held out for testing. Unique athlete IDs ensured no overlap between sets.

B. Class Imbalance Strategies

The dataset exhibited severe class imbalance, with only 14.6 percent of entries belonging to medalists, so two resampling methods were applied to the training data:

- SMOTE: creates synthetic minority samples using nearest neighbours.
- Random undersampling: removes majority-class cases until classes are balanced.

Both resampled sets were evaluated on the untouched 2016 test set. Fig. 2 shows the post-resampling distributions.

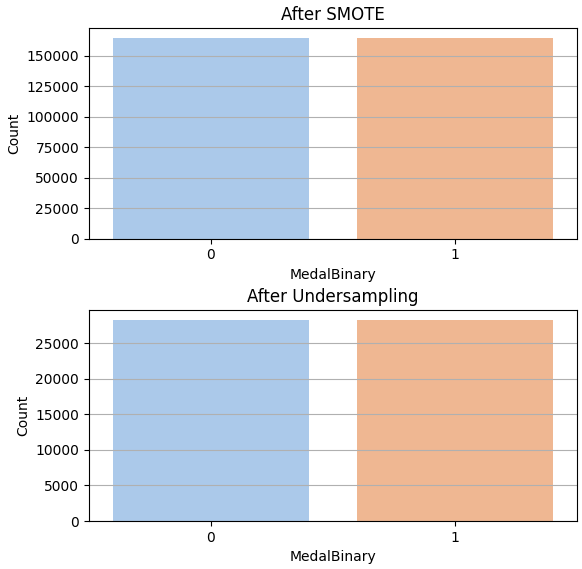


Fig. 2. Class distributions

C. Algorithms and Hyperparameter Tuning

Three supervised learners were selected to ensure diversity in learning behavior and performance under class imbalance. Logistic Regression provides a linear baseline with high recall. Random Forest and XGBoost are both tree-based, but they differ in their ensemble strategies, using bagging and boosting respectively. RandomizedSearchCV with three-fold cross-validation was used for efficient hyperparameter tuning. The final model configurations are presented in Table II.

TABLE II. BEST HYPERPARAMETERS FOR EACH MODEL

Model	Key Parameters
Logistic Regression	C = 0.1, penalty = l2
Random Forest	n_estimators = 300, max_depth = 10, min_samples_split = 5
XGBoost	n_estimators = 200, max_depth = 6, learning_rate = 0.1

Every model was trained twice, once on the SMOTE set and once on the undersampled set, so any change in performance could be tied directly to the resampling choice.

D. Stacking Ensemble

To improve generalisation, a stacking ensemble was constructed using Logistic Regression, Random Forest, and XGBoost as base learners. Their predicted probabilities were used as input features to a meta-learner, which was trained to optimise model combination. Logistic Regression was chosen as the final estimator. Stacking enables the ensemble to learn how to weight each base model based on its individual strengths rather than treating all equally. Separate

ensembles were trained for the SMOTE and undersampling conditions to maintain consistency in evaluation.

All models, including the ensemble, were later evaluated on the same 2016 hold-out test set.

IV. EXPERIMENTAL SETUP

All experiments were implemented in Python 3.11 using pandas 2.3.1, scikit-learn 1.6.1, imbalanced-learn 0.13.0, XGBoost 3.0.2, and SHAP 0.48.0.

A. Data Processing

Resampling (SMOTE or random undersampling) was applied only to the training set to avoid bias. Test data remained in its original class distribution. Numerical features were standardised with StandardScaler; the scaler was fitted on the resampled training data and then applied to both training and test sets. This approach preserves time consistency and avoids look-ahead bias [6].

B. Model Training

Each classifier, Logistic Regression, Random Forest, and XGBoost, was tuned with RandomizedSearchCV (3-fold cross-validation, stratified by class). The same random seed (15) was used across all libraries to ensure reproducibility. Final models were retrained on the full resampled training data with the best hyperparameters identified during tuning.

C. Evaluation Metrics

Performance was assessed on the 2016 hold-out set using five metrics:

- Accuracy: proportion of correct predictions
- Precision: percentage of predicted medalists that were correct
- Recall: percentage of actual medalists identified
- F1 Score: harmonic mean of precision and recall
- ROC AUC: ranking capability across thresholds

These metrics provide complementary views of performance under class imbalance. Reported metrics are point estimates based on predictions on the 2016 hold-out test set.

V. RESULTS

A. Quantitative Metrics

Table III summarises the performance of all models under both SMOTE and random undersampling (US). The results indicate a trade-off between raw accuracy and recall-based metrics.

XGBoost with SMOTE achieves the highest accuracy (0.830) but shows a low recall (0.176), reflecting poor sensitivity to medalists. Logistic Regression with SMOTE delivers the highest recall (0.699) but exhibits low precision (0.162), resulting in a high number of false positives.

XGBoost trained on the undersampled data provides the best overall balance, attaining the highest F1 Score (0.308) and ROC AUC (0.702). Stacking with undersampling achieved the best balance overall, matching XGBoost (US) in both F1 (0.308) and AUC (0.702), and improving recall. This confirms the benefit of adaptive model combination.

TABLE III. PERFORMANCE METRICS OF ALL MODELS

Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
Logistic Regression (SMOTE)	0.505	0.162	0.699	0.264	0.619
Random Forest (SMOTE)	0.816	0.222	0.180	0.199	0.639
XGBoost (SMOTE)	0.830	0.253	0.176	0.208	0.670
Stacking (SMOTE)	0.830	0.242	0.161	0.193	0.653
Logistic Regression (US)	0.509	0.162	0.690	0.263	0.617
Random Forest (US)	0.558	0.187	0.740	0.298	0.685
XGBoost (US)	0.558	0.192	0.774	0.308	0.702
Stacking (US)	0.553	0.192	0.786	0.308	0.702

B. ROC Curves

Fig. 3 shows the Receiver Operating Characteristic (ROC) curves for each model–resampling combination. XGBoost with undersampling yields the largest area under the curve, indicating the strongest ranking ability. The undersampled Stacking Ensemble follows closely.

In contrast, SMOTE-trained models exhibit greater variation: Logistic Regression favours recall but sacrifices precision, while tree-based models lean toward the majority class. These patterns confirm that undersampling improves balanced detection of medalists.

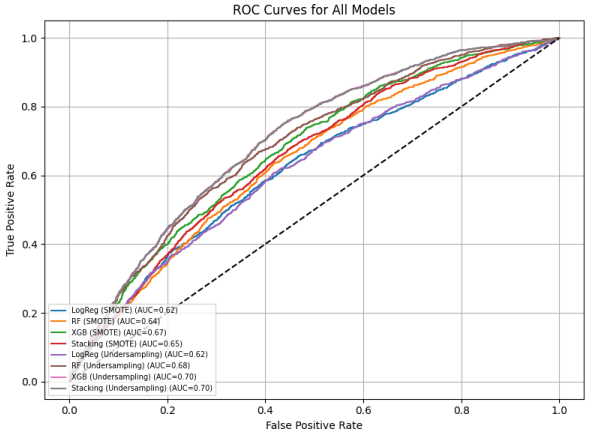


Fig. 3. ROC Curves of all models on the 2016 test set.

C. Confusion Matrices

Fig. 4 presents the confusion matrices for SMOTE-trained models. Random Forest and XGBoost correctly identify most non-medalists but miss many medalists, consistent with their low recall. Logistic Regression captures more medalists but produces numerous false positives.

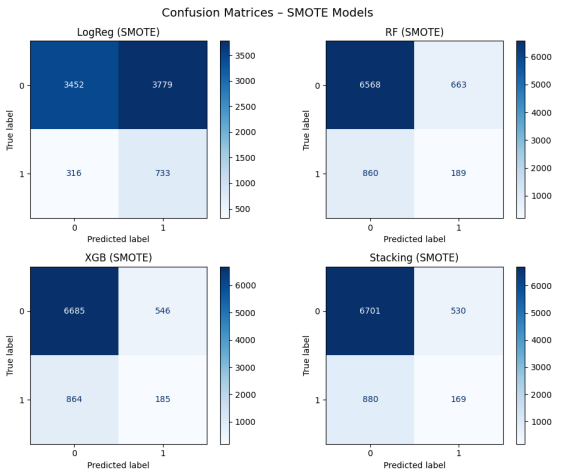


Fig. 4. Confusion Matrices for SMOTE

Fig. 5 shows the matrices for undersampling. XGBoost and the Stacking Ensemble achieve the best trade-off, increasing true positives while containing false positives. Random Forest also improves its recall under undersampling. These results reinforce that undersampling enhances minority class detection for tree-based models.

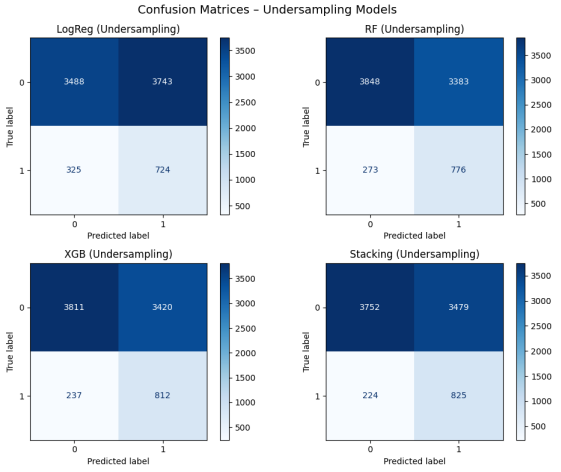


Fig. 5. Confusion Matrices for Undersampling

D. Feature Importance

Fig. 6 presents the SHAP summary for XGBoost on the undersampled data.

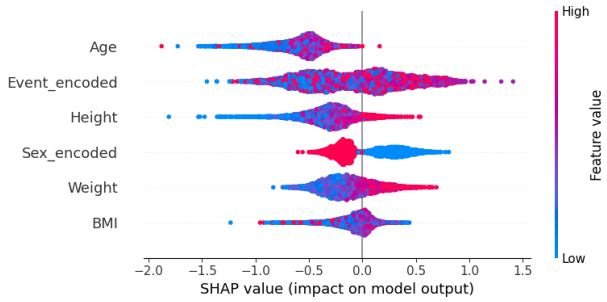


Fig. 6. SHAP summary plot for XGBoost

SHapley Additive exPlanations (SHAP) identified Age and Event_encoded as the most influential features, followed by Height and Sex_encoded. The importance of Sex_encoded justifies its inclusion, as many events are gender-specific or shared. Male athletes and those in prime-age ranges had higher SHAP values. Event_encoded carried signal but suffered from high cardinality. Overall, basic demographic traits were strong predictors, though domain-specific features could improve performance.

VI. DISCUSSION

Addressing class imbalance had a marked effect on model effectiveness. On the undersampled data, XGBoost (US) achieved the highest F1 score (0.308) and ROC-AUC (0.702). Its confusion matrix showed the most favorable balance between true-positive and false-positive rates, indicating that reducing the majority class helped the classifier recognise minority cases, in line with earlier work on undersampling [7]. The Stacking Ensemble (US) performed similarly, matching the best base model in F1 and AUC while delivering the highest recall (0.786), highlighting the value of combining diverse learners through a meta-model. Soft voting was also tested but produced slightly lower F1 and recall, particularly on the undersampled data, so stacking was retained as the preferred ensemble technique.

SMOTE yielded a different pattern. Although raw accuracy improved for the tree-based models, recall declined, echoing the findings of Douzas and Bacao that synthetic samples may still reinforce majority trends when classes have high feature overlap [8]. Logistic Regression with SMOTE showed the opposite tendency, achieving very high recall but extremely low precision, which is a common trade-off for linear models on skewed data.

SHAP analysis agreed with these quantitative results. Age and Event_encoded were the most influential predictors, followed by Height and Sex_encoded, while BMI contributed least. The prominence of Sex_encoded supports its inclusion with Event_encoded, because many events are gender-specific or shared across genders. The modest impact of Event_encoded suggests that collapsing numerous event categories into a single feature may obscure useful context. Previous studies report that demographic variables alone can be sufficient in sports such as basketball [10], and that high-

cardinality encodings may reduce accuracy [9].

Several limitations remain. The present models do not incorporate dynamic performance indicators such as recent form, which could improve predictive accuracy. In addition, event types are collapsed into a single categorical variable, which limits contextual detail. Future work should investigate temporal variables, hierarchical event encodings, and adaptive resampling that balances classes within sport-specific subsets [6]. Integration with athlete-tracking platforms used by national federations could allow early identification of medal potential and guide strategic resource allocation.

Before deployment, ethical considerations must be addressed. Attributes such as sex and age can encode historical bias, and reliance on archival data could perpetuate structural inequities. Any application that influences team selection therefore requires rigorous fairness audits, transparent methods, and compliance with data-protection frameworks such as the GDPR.

VII. CONCLUSION

This study benchmarked Logistic Regression, Random Forest, and XGBoost under two resampling strategies to predict Olympic medalists from historical athlete data. XGBoost trained on undersampled data delivered the best performance (F1 = 0.308; ROC-AUC = 0.702), underscoring the importance of handling class imbalance. Ensemble learning also proved effective: the stacking classifier matched the best single model and recorded the highest recall on the undersampled set. SHAP analysis identified age and event type as the most informative features.

These findings demonstrate that simple demographic and event-based information can support medal-prediction models. Future studies should incorporate dynamic variables, multi-class outcomes, and fairness-aware design. Operational deployment will require robust validation, transparency, and ethical safeguards to ensure responsible use in elite-sport decision-making.

VIII. REFERENCES

[1] M. Cordeiro, F. S. Oliveira, G. Resende, and J. M. Pinto, "A synthetic data-driven machine learning approach for predicting physical performance in sports," Frontiers in Sports and Active Living, vol. 7, 2025. [Online]. Available: https://www.frontiersin.org/articles/10.3389/fspor.2025.1607600/full

[2] M. Mujahid, "Data oversampling and imbalanced datasets: an investigation," Journal of Big Data, vol. 11, no. 1, Jan. 2024. [Online]. Available: https://journalofbigdata.springeropen.com/articles/10.1186/s40537-024-00943-4

[3] Z. Zhang, X. Sun, and M. Yu, "Predicting Olympic medal performance for 2028 using demographic and historical patterns," Applied Sciences, vol. 15, no. 14, 2025, Art. no. 7793. [Online]. Available: https://www.mdpi.com/2076-3417/15/14/7793

[4] C. Schlembach, D. Müller, and M. Sutter, "Forecasting the Olympic medal distribution during a pandemic: A socio-economic machine learning model," SSRN Electronic Journal, 2020. [Online]. Available: https://www.researchgate.net/publication/347590215

[5] J. H. Joloudari et al., "Effective class-imbalance learning based on SMOTE variants and ensemble techniques," Applied Sciences, vol. 13, no. 6, 2023, Art. no. 4006. [Online]. Available: https://www.mdpi.com/2076-3417/13/6/4006

[6] V. Cerqueira, L. Torgo, and R. Pfahringer, "Insights into performance evaluation for time series forecasting," arXiv preprint arXiv:2008.06899, 2020. [Online]. Available: https://arxiv.org/abs/2008.06899

[7] Y. Roumani, J. Li, and S. Mandell, "Sports analytics in the NFL: Classifying offensive strategies while handling imbalanced data," PLoS ONE, vol. 17, no. 12, Dec. 2022, Art. no. e0278725. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9684891/

[8] N. Li et al., "Hybrid intelligent model for Olympic medal prediction using Bayesian optimisation and XGBoost," Technologies, vol. 13, no. 6, 2025, Art. no. 250. [Online]. Available: https://www.mdpi.com/2227-7080/13/6/250

[9] S. Chiang and G. Denes, "Supervised learning for table tennis match prediction," arXiv preprint arXiv:2303.16776, 2023. [Online]. Available: https://arxiv.org/abs/2303.16776

[10] C. B. Taber, S. Sharma, M. S. Raval, et al., "A holistic approach to performance prediction in elite women's basketball," Scientific Reports, vol. 14, 2024, Art. no. 1162. [Online]. Available: https://www.nature.com/articles/s41598-024-51658-8

IX. APPENDIX

A. Dataset

The dataset used in this project is publicly available from Kaggle:

- Title: 120 Years of Olympic History: Athletes and Results
- Source: https://www.kaggle.com/datasets/heesoo37/120-years-of-olympic-history-athletes-and-results
- File Used: athlete_events.csv

B. Project Code

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import shap
import json

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, VotingClassifier,
StackingClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, ConfusionMatrixDisplay
)
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
from xgboost import XGBClassifier

# load dataset
df = pd.read_csv('athlete_events.csv')
print("Initial dataset size:", df.shape)

# create binary label
df['MedalBinary'] = df['Medal'].notnull().astype(int)

# drop rows with missing values
df.dropna(subset=['Age', 'Height', 'Weight', 'Sex', 'Sport', 'Event'],
inplace=True)
print("After dropping missing values:", df.shape)

# create features
df['BMI'] = df['Weight'] / ((df['Height'] / 100) ** 2)
df['Sex_encoded'] = LabelEncoder().fit_transform(df['Sex'])
df['Event_encoded'] = LabelEncoder().fit_transform(df['Event'])

features = ['Age', 'Height', 'Weight', 'BMI', 'Event_encoded', 'Sex_encoded']
df = df.copy()


# split train/test
train_df = df[df['Year'] < 2016]
test_df = df[df['Year'] == 2016]
train_ids = set(train_df['ID'])
test_df = test_df[~test_df['ID'].isin(train_ids)]

X_train = train_df[features]
y_train = train_df['MedalBinary']
X_test = test_df[features]
```

```
y_test = test_df['MedalBinary']



# resampling
X_train_smote, y_train_smote = SMOTE(random_state=15).fit_resample(X_train,
y_train)
X_train_under, y_train_under =
RandomUnderSampler(random_state=15).fit_resample(X_train, y_train)



# plot class distributions
fig, axes = plt.subplots(3, 1, figsize=(6, 10))
sns.countplot(x=y_train, ax=axes[0], palette='pastel')
axes[0].set_title("Before Resampling")
sns.countplot(x=y_train_smote, ax=axes[1], palette='pastel')
axes[1].set_title("After SMOTE")
sns.countplot(x=y_train_under, ax=axes[2], palette='pastel')
axes[2].set_title("After Undersampling")
for ax in axes:
    ax.set_xlabel("MedalBinary")
    ax.set_ylabel("Count")
    ax.grid(axis='y')
plt.suptitle("Class Distribution Before and After Resampling", fontsize=14)
plt.tight_layout()
plt.show(block=False)



# load tuned hyperparameters
with open("best_model_params.json") as f:
    best_params = json.load(f)

results = []
f1_scores = []
fitted_models = {}
colors = sns.color_palette("tab10", 10)
color_idx = 0
X_test_scaled_map = {}

plt.figure(figsize=(8, 6))

resamplers = {
    'SMOTE': (X_train_smote, y_train_smote),
    'Undersampling': (X_train_under, y_train_under)
}

for method_name, (X_res, y_res) in resamplers.items():
    scaler = StandardScaler()
    X_res_scaled = scaler.fit_transform(X_res)
    X_test_scaled = scaler.transform(X_test)
    X_test_scaled_map[method_name] = X_test_scaled

    logreg =
LogisticRegression(**best_params[f'LogisticRegression_{method_name}'],
class_weight='balanced')
    rf = RandomForestClassifier(**best_params[f'RandomForest_{method_name}'],
class_weight='balanced', random_state=15)
    xgb = XGBClassifier(**best_params[f'XGBoost_{method_name}'],
use_label_encoder=False, eval_metric='logloss', random_state=15)

    stacking = StackingClassifier(
```

```
        estimators=[('lr', logreg), ('rf', rf), ('xgb', xgb)],
        final_estimator=LogisticRegression(),
        cv=3,
        passthrough=False
    )

    models = {
        'LogReg': logreg,
        'RF': rf,
        'XGB': xgb,
        'Stacking': stacking
    }

    for key, model in models.items():
        label = f'{key} ({method_name})'
        model.fit(X_res_scaled, y_res)
        y_pred = model.predict(X_test_scaled)
        y_proba = model.predict_proba(X_test_scaled)[:, 1]

        results.append({
            'Model': label,
            'Accuracy': accuracy_score(y_test, y_pred),
            'Precision': precision_score(y_test, y_pred),
            'Recall': recall_score(y_test, y_pred),
            'F1 Score': f1_score(y_test, y_pred),
            'ROC AUC': roc_auc_score(y_test, y_proba)
        })

        fitted_models[label] = model
        f1_scores.append((label, f1_score(y_test, y_pred)))

        fpr, tpr, _ = roc_curve(y_test, y_proba)
        plt.plot(fpr, tpr, label=f'{label} (AUC={roc_auc_score(y_test,
y_proba):.2f})', color=colors[color_idx])
        color_idx += 1



# evaluation results
result_df = pd.DataFrame(results)
print("\n==== Evaluation Metrics Summary ====")
print(result_df.round(3))



# ROC curve
plt.plot([0, 1], [0, 1], 'k--')
plt.title("ROC Curves for All Models")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend(loc='lower left', fontsize=7)
plt.grid()
plt.tight_layout()
plt.show(block=False)



# confusion matrices SMOTE
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
for ax, label in zip(axes.flat, [k for k in fitted_models if '(SMOTE)' in k]):
    ConfusionMatrixDisplay.from_predictions(
        y_test,
        fitted_models[label].predict(X_test_scaled_map['SMOTE']),
```

```
        ax=ax, cmap='Blues'
    )
    ax.set_title(label)
plt.suptitle("Confusion Matrices - SMOTE Models", fontsize=14)
plt.tight_layout()
plt.show(block=False)



# confusion matrices - undersampling
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
for ax, label in zip(axes.flat, [k for k in fitted_models if '(Undersampling)'
in k]):
    ConfusionMatrixDisplay.from_predictions(
        y_test,
        fitted_models[label].predict(X_test_scaled_map['Undersampling']),
        ax=ax, cmap='Blues'
    )
    ax.set_title(label)
plt.suptitle("Confusion Matrices - Undersampling Models", fontsize=14)
plt.tight_layout()
plt.show(block=False)


# SHAP on XGB (SMOTE)
scaler_final = StandardScaler()
X_train_scaled = scaler_final.fit_transform(X_train_smote)
X_test_scaled = scaler_final.transform(X_test)

xgb_model = XGBClassifier(**best_params['XGBoost_SMOTE'],
use_label_encoder=False, eval_metric='logloss', random_state=15)
xgb_model.fit(X_train_scaled, y_train_smote)

explainer = shap.TreeExplainer(xgb_model)
shap_values = explainer.shap_values(X_test_scaled)

plt.figure()
shap.summary_plot(shap_values, X_test, feature_names=X_train.columns,
show=False)
plt.tight_layout()
plt.show(block=False)

input("Stop")
```